Kevin Küster

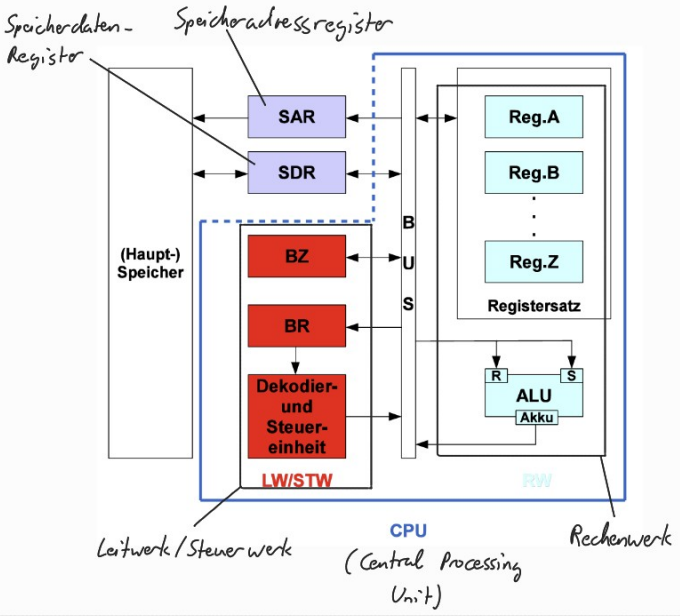
Zusammenfassung: RARCH

# RARCH

## Von - Neumann - Rechner

### Elemente

1. Rechenwerk
   ↳ arithmetische logische Operationen
2. Steuerwerk (Leitwerk)
   ↳ Steuert Abläufe
3. Speicherwerk
   ↳ Hauptspeicher & Arbeitsspeicher
4. Ein-/Ausgabewerk




### Befehlsphasen

1. BH: Befehlshol-phase
2. DE: Dekodierphase
3. OH: Operandenhol-phase
4. AP: Ausführungsphase
5. RS: Rückschreibphase
6. AD: Adressierungsphase

1 Befehl : 8 Takte

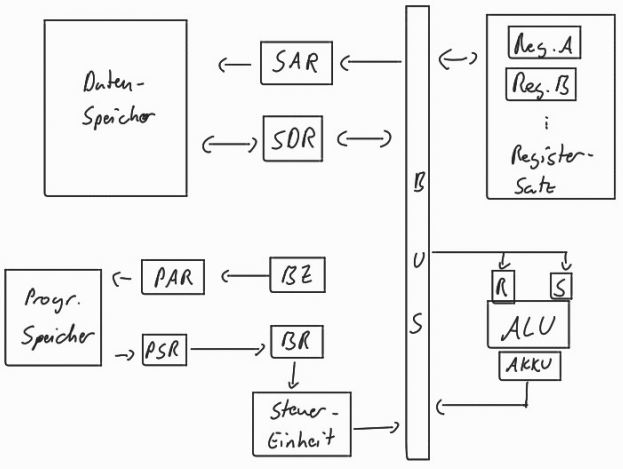
## Von-Neumann - Bottleneck

Daten & Programme (Befehle) stehen im selben Speicher (Hauptspeicher)

=> Zugriff erfolgt für alle Anfragen auf dem gleichen Bus => Effizienzverlust

## Harvard - Rechner

Getrennte Speicher für Daten und Programme:




## Beispiel

Register - Transfer - Notation:

[ ] ≙ Wert, [[ ]] ≙ Referenzierwert, ( ) ≙ Teil, Ziel ⟵ Herkunft

3 - Adress - Befehl: [Add Reg.B, Mem.X, Reg.F]

Adresse, Daten

1. BH: [SAR] ⟵ [BZ]
   [SDR] ⟵ [[SAR]] (Adresse im Hauptspeicher)
   [BR] ⟵ [SDR]
2. DE: Kein Datenaustausch zwischen Speichern
3. OH: [SAR] ⟵ [BR (drittes Bit)]
   SDR ⟵ [[SAR]]
   Reg.A ⟵ [SDR] (Erstes freies Register)
4. AP: ALU.R ⟵ [Reg.A]
   ALU.L ⟵ [Reg.X]
   ALU.AKKU ⟵ [ALU.L] + [ALU.R]
5. RS: Reg.B ⟵ [ALU.AKKU]
6. AD: => Inkrementierung BZ: ALU.R ⟵ [BZ]
   ALU.AKKU ⟵ [ALU.R] + 1
   BZ ⟵ [ALU.AKKU]

## Klassifikationen

### Flynn

**Single Instruction Single Data (SISD)**

Pro Prozessortakt: Eine Instruktion auf ein Datenwort

**Single Instruction Multi Data (SIMD)**

Pro Prozessortakt: Eine Instruktion auf mehrere Datenwörter gleichzeitig

**Multi Instruction Single Data (MISD)**

Pro Prozessortakt: Mehrere Instruktionen auf ein Datenwort

**Multi Instruction Multi Data (MIMD)**

Pro Prozessortakt: Mehrere Instruktionen auf mehrere Datensätze gleichzeitig

---

### Erlanger

**Klassifikation**

$\epsilon = (k \circ k', d \circ d', w \circ w')$

$k$: Anzahl Steuerwerke / Leitwerke

($k'$: Steuerwerke Prozessor-Pipelining)

$d$: Anzahl Rechenwerke je Steuerwerk

($d'$: Pipelining-Rechenwerke je Steuerwerk)

$w$: Anzahl parallele Bitstellen / Wortbreite (ALU)

$w'$: elementare Teilstufen (Pipelinestufen)

**Erlanger → Flynn**

$(1, 1, x) \Rightarrow$ SISD

$(1, >1, x) \Rightarrow$ SIMD

$(>1, x, y) \Rightarrow$ MIMD

MISD $\Rightarrow$ unrealistisch

---

### Gilo i

**Impliziierter Parallelismus**

Nutzung der in den Programmen gegebenen Möglichkeiten zur parallelen Ausführung

**Explizierter Parallelismus**

Parallele Ausführung vorbestimmt durch Anlage von Programmen & Datenstrukturen

**Homogene Multiprozessoren**

Alle CPUs sind vom gleichen Typ und identisch konfiguriert

**Inhomogene Multiprozessoren**

Rechnerkerne unterscheiden sich: Leistungsfähigkeit, Funktion- & Befehlsumfang, Befehlsstruktur, Konfiguration

---

### Benes-Netzwerke

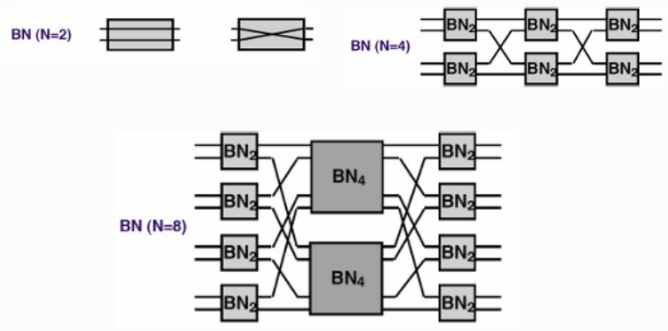
dynamisch, rekursiv definiertes Netzwerk

Grundelement: 2x2 Kreuzschienenverteiler (crossbar)

↳ $N = 2^a$ Eingänge → $2 \cdot (\log_2 N) - 1$ Stufen

↳ Jede Stufe: $N/2$ Crossbars

↳ BN: Insgesamt $N \cdot (\log_2 N - \frac{1}{2})$ Crossbars



## Kodierung

**8-4-2-1, 2-4-2-1**

Ordnet jeder Dezimalstelle einen 4 Bit großen Block zu

**Beispiel**

Dezimal: $192_{10}$

Dual: 11000000

8-4-2-1: 0001 1001 0010

2-4-2-1: 0001 1111 0010

**Gray-Code**

| | |
|---|---|
| 0 - 0000 | 5 - 0111 |
| 1 - 0001 | 6 - 0101 |
| 2 - 0011 | 7 - 0100 |
| 3 - 0010 | 8 - 1100 |
| 4 - 0110 | 9 - 1101 |

## Hexadezimal

Basis: 16
0 - F

| Dezimal | Binär | Hex |
|---|---|---|
| 0 | 0000 | 0 |
| 1 | 0001 | 1 |
| 2 | 0010 | 2 |
| 3 | 0011 | 3 |
| 4 | 0100 | 4 |
| 5 | 0101 | 5 |
| 6 | 0110 | 6 |
| 7 | 0111 | 7 |
| 8 | 1000 | 8 |
| 9 | 1001 | 9 |
| 10 | 1010 | A |
| 11 | 1011 | B |
| 13 | 1100 | C |
| 14 | 1101 | D |
| 15 | 1110 | E |
| 16 | 1111 | F |

## Redundanz

$R = m_p - m_s$

$m_p$: Entscheidungsgehalt des physikalischen Wortsatzes
→ Anzahl aller möglichen ($2^n$)

$m_s$: Entscheidungsgehalt des semantischen Wortsatzes
→ Anzahl der vorhandenen

**Beispiel**

5000 Wörter, Kodierung: $16_{10}$ Bit:

$m_s = \log_2(5000) = \log_2(5) + \log_2(1000)$

$\approx 2{,}287 + 10 = 12{,}287$

$m_p = 16$, $R = 16 - 12{,}287 \approx 3{,}7$

## Stellen- & Hamming-Distanz

Stellendistanz: Anzahl der unterschiedlichen Bits zwischen Codewörtern

Hamming-d. (D): kleinste Stellendistanz im gesamten Code

Fehlererkennung: $F_E = D - 1$, Fehlerkorrektur: $F_k = \frac{D-1}{2}$

**Beispiel**

a: $00000_2$, b: $00111_2$, c: $11001_2$, d: $11110_2$

Stellendistanzen: (a,b): 3, (a,c): 3, (a,d): 4, (b,c): 3
(b,d): 3, (c,d): 3 ⇒ $\underline{\underline{D = 3}}$

$F_E = 3 - 1 = 2$ ⇒ 1 Bit & 2 Bit Fehler

$F_k = \frac{3-1}{2} = 1$ ⇒ 1 Bit Fehler

⇒ Setzt voraus, dass 1 Bit Fehler wahrscheinlicher sind
↳ Keine sichere Korrekturmöglichkeit

## Codesicherung - Quersummenprüfung

Paritätsbit: Anfügen eines Paritätsbits am Ende des Codewortes zur gewährleistung einer geraden (Even Parity) bzw. ungeraden (Odd Parity) Quersumme einschließlich Paritybit

**Beispiel**

| Dezimal | 8-4-2-1 | Even P. | Odd P. |
|---|---|---|---|
| 0 | 0000 | 0 | 1 |
| 1 | 0001 | 1 | 0 |
| 2 | 0010 | 1 | 0 |
| 3 | 0011 | 0 | 1 |
| 4 | 0100 | 1 | 0 |
| 5 | 0101 | 0 | 1 |

## Komplementbildung

Darstellung: -x, **Beispiel**: $-99_{10}$

$|x|$: $|-99_{10}| = 99_{10} = 1100011_2$

8 Bit Format: 01100011

Einerkomplement: 10011100

Zweierkomplement: EK +1 = 10011101

Rekomplementierung: 10011101

1.) Invertieren → 01100010

2.) +1 → 01100011

Erstes Bit als Vorzeichen:
0-255 → -127 - 127

## Umwandlung - Horner

Dezimal: $\dots 10^3, 10^2, 10^1, 10^0 \mid 10^{-1}, 10^{-2}, \dots$

Dual: $\dots 2^3, 2^2, 2^1, 2^0 \mid 2^{-1}, 2^{-2}, 2^{-3}, \dots$

### Dezimal → Dual

**Beispiel**: 196,23

1.) Zerlegung in Ganzzahl und Nachkomma Anteil:

196,23 → 196 + 0,23

2.) Ganzzahlige Division mit Rest

⇒ Durch Basis teilen, Dual → Basis 2:

196/2 = 98 R 0 ↑
98/2 = 49 R 0
49/2 = 24 R 1
24/2 = 12 R 0
12/6 = 6 R 0
6/2 = 3 R 0
3/2 = 1 R 1
1/2 = 0 R 1

$196_{10} = 11000100_2$ →

3.) Nachkommateil mit Basis Multiplizieren

Rest: Ganzzahliger Anteil der Multiplikation

Ergebnis: Neuer Nachkommteil

→ Bis Rest 0 oder gewünschte Genauigkeit

0,23 · 2 = 0,46 R 0
0,46 · 2 = 0,92 R 0
0,92 · 2 = 0,84 R 1
0,84 · 2 = 0,68 R 1
0,68 · 2 = 0,36 R 1
0,36 · 2 = 0,72 R 0
0,72 · 2 = 0,44 R 1
0,44 · 2 = 0,88 R 0 ↓

$0,23_{10} = 0,00111010_2$

4.) Ergebnis zusammenführen

192,23 ≈ 1100 0100, 0011 1010

### Dual → Dezimal

**Beispiel**: $10111111,00111101_2$

1.) Zerlegung in Ganzzahl und Nachkomma Anteil:

⇒ 10111111 + 00111101

2.) Iterative Multiplikation → Ganzzahl

← ..... ←

1 0 1 1 1 1 1 1
↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
$2^7$ $2^6$ $2^5$ $2^4$ $2^3$ $2^2$ $2^1$ $2^0$

$\Rightarrow 2^7 + 2^5 + 2^4 + 2^3 + 2^2 + 2 + 1$

$= 128 + 32 + 16 + 8 + 4 + 2 + 1 = 191$

3.) Iterative Division → Nachkommateil

→ .....

0 0 1 1 1 1 0 1
↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
$2^{-1}$ $2^{-2}$ $2^{-3}$ $2^{-4}$ $2^{-5}$ $2^{-6}$ $2^{-7}$ $2^{-8}$

$\Rightarrow 2^{-3} + 2^{-4} + 2^{-5} + 2^{-6} + 2^{-8} \approx 0,23828125$

0 0 1 1 1 1 0 1

1 + 0 /2 = 0,5
0 + 0,5 /2 = 0,25
1 + 0,25 /2 = 0,625
1 + 0,625 /2 = 0,8125
1 + 0,8125 /2 = 0,90625
1 + 0,90625 /2 = 0,953125
0 + 0,953125 /2 = 0,4765625
0 + 0,4765625 /2 = 0,23828125

4.) Ergebnis zusammenführen

$10111111,00111101_2 = 191,23828125_{10}$

## Umwandlung - Euklid

**Beispiel**: $168,625_{10}$ → Basis 4

1.) Anfangspotenz ermitteln - Ganzzahl

$4^3 = 64 < 168 < 4^4 = 256 \Rightarrow 4$

2.) Iterative Division

168 / $4^4$ = 0 R 168
168 / $4^3$ = 2 R 40
40 / $4^2$ = 2 R 8
8 / $4^1$ = 2 R 0
0 / $4^0$ = 0 R 0 ↓

$168_{10} = 2220_4$

3.) Iterative Divison - Nachkommateil

0,625 / $4^{-1}$ = 0,625 · 4 = 2 R 0,125 ↓
0,125 / $4^{-2}$ = 0,125 · 16 = 2 R 0

$0,625_{10} = 0,22_4$

4.) Ergebnis Zusammenführen

$168,625_{10} = 2220,22_4$

# Gleitkommazahlen

## 754r, 16-Bit

Vorzeichen | Exponent + BIAS | Mantisse
[1 Bit] [5 Bit] [10 Bit]

**Beispiel**: 48,625

1.) Dualsystem:

$48_{10} = 110000_2$

$0{,}625_{10} = 0{,}101_2$

$\Rightarrow 110000{,}101_2$

2.) Normieren

$110000{,}101_2 = 1{,}10000101 \cdot 2^5$

3.) Exponent

Exp. = 5, BIAS = 15 $\Rightarrow 20_{10}$

$\hookrightarrow 10100_2$

4.) Vorzeichen

Positiv $\Rightarrow 1$

5.) ggf. Auf 16 Bit auffüllen

1|10100|1000010100

$\Rightarrow 1|10100|1000010100_2$

## Rechenoperationen

$X + Y = m_x \cdot 2^{e_x} + m_y \cdot 2^{e_y}$

$= (m_x \cdot 2^{e_x - e_y} + m_y) \cdot 2^{e_y}, e_x \leq e_y$

$X - Y = m_x \cdot 2^{e_x} - m_y \cdot 2^{e_y}$

$= (m_x \cdot 2^{e_x - e_y} - m_y) \cdot 2^{e_y}, e_x \leq e_y$

$X \cdot Y = (m_x \cdot m_y) \cdot 2^{e_x + e_y}$

$\frac{X}{Y} = \left(\frac{m_x}{m_y}\right) \cdot 2^{e_x - e_y}$

$\Rightarrow$ Mantissen mit führender 1

## Addition

1. $x + y$, $x > 0$, $y > 0$

```
 0111    7
+0100   +4
------  --
 1011   11
```

2. $x + y$, $x > 0$, $y < 0$, $|x| > |y|$

```
 00001011    11
+11111010   +(-6)
--------    ---
1|00000101    9
```

↑ Übertrag wird verworfen

$\Rightarrow$ Komplementbildung

3. $x + y$, $x > 0$, $y < 0$, $|x| < |y|$

```
 00010000    16
+11101000   -24
--------    ---
 11111000    -8
```

↳ Ergebnis im Zweierkomplement

4. $x + y$, $x < 0$, $y < 0$

```
 111111011    -5
+111110111   +(-9)
---------    ---
1|11110010   -14
```

↳ Ergebnis im Zweierkomplement

## Subtraktion

$x - y = x + (-y)$, siehe Addition

bzw.

$0 - 0 = 0$

$1 - 0 = 1$

$1 - 1 = 0$

$0 - 1 = 1 +$ Übertrag

Randfälle (Übertrage):

```
 0    0    1
-1   -1   -1
---  ---  ---
 1    0    1
```

**Beispiel**

```
 1100
-1011
------
 0001
```

## Multiplikation

Multiplikation mittels Partialprodukten

```
 1110 · 110
 ------
 1110
  1110
+ 10000
 -------
 1011100
```

## Division

Äquivalent zum Dezimalsystem:

```
 11101 / 101 = 101 R 100
-101 ↓
 0100
    0 ↓
  1001
 -101
  0100
```

$\Rightarrow 29 : 5 = 5$ R 4

---

**Beispiel**

$a = 512{,}5_{10}$, $b = 4{,}625_{10}$

$= 1|11000|0000000001_2$ $= 0|10001|00101000000_2$

$a + b = b + a = 1{,}00101 \cdot 2^2 + 1{,}0000000001 \cdot 2^9$

$= (m_b \cdot 2^{e_b - e_a} + m_a) \cdot 2^{m_a}$

$= (1{,}00101 \cdot 2^{2-9} + 1{,}0000000001) \cdot 2^9$

```
 = 0,0000 0010 0101   ↝ Übercarry (>10 Bit) wird
+1,0000 0000 01        verworfen
--------------
 1,0000 0010 10
```

Exp = max {2, 9} = 9

$\Rightarrow$ ggf. Anpassung des Exponenten aufgrund von Übercarry: ✓

Vorzeichenbetrachtung: 0

$\Rightarrow 0|11000|0000001010$

$a \cdot b = (m_x \cdot m_y) 2^{e_x + e_y}$

$= (1{,}0000000001 \cdot 1{,}00101) \cdot 2^{2+9}$

Exp = 11 (ohne Bias)

1,0000000001 · 1,00101

```
 1,0000 0000 01
+0· 0,1000 0000 001
+0· 0,0100 0000 0001
+1· 0,0010 0000 0000 1
+0· 0,0001 0000 0000 01
+1· 0,0000 1000 0000 001
-----------------------
 1,0010 1000 01
```

Vorzeichen mittels XOR: 0 xor 0 = 0

ggf. Anpassung Exponent: ✓

$\Rightarrow 0|11010|0010100001$

## n - Adressmaschinen

**Beispiel:** $z = x - y$

**3 - Adress:** OP-Code | Ziel | $Op_1$ | $Op_2$

sub z x y | $z = x - y$

**2 - Adress:** OP-Code | Ziel / $Op_1$ | $Op_2$

sub x y | $x = x - y$

mov z x | $z = x$

**1,5 - Adress:** OP-Code | Ziel / $Op_1$ | $Op_2$

=> In jeder Operation liegt min. 1 Operand in einem Register vor

mov h1 x | h1 (Hilfsreg. 1) = x

sub h1 y | h1 = h1 - y

mov z h1 | z = h1

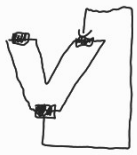
**1 - Adress:** OP-Code | $Op_1$

=> Berechnung via Akku (ALU)

Load x | Akku = x

sub y | Akku = Akku - y

stor z | z = Akku



**0 - Adress:** Stackoperation

=> Stack: | y | x |

push x | x

push y | x y

sub | x y - => x - y

pop z | z = x - y

0-Adr.-M.: Geschw.: sehr hoch / Befehlsanzahl: sehr viele => Benötigt mehr speicher, Fehleranfälliger

1-Adr.-M.: Geschw.: hoch / Befehlsanzahl: viele

1,5-Adr.-M.: Geschw.: hoch / Befehlsanzahl: viele

2-Adr.-M.: Geschw.: mittel / Befehlsanzahl: wenige

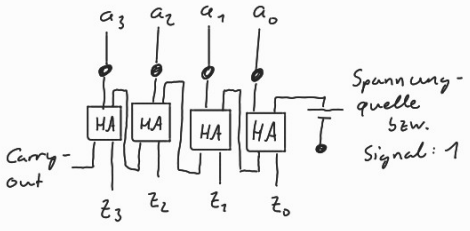
3-Adr.-M.: Geschw.: gering / Befehlsanzahl: sehr wenige => Weniger Speicher, Einfache(re) Implementierung

---

## ALU

### Halbaddierer

Zweier-komplement 4-Bit-Zahl:




Carry ≙ Übertrag




| x | y | c | s |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 |

### Aufbau + FLags

Flags => Steuerung des Programms

Negative-Flag: Vorzeichen -> 1 ≙ -, 0 ≙ +

Zero-Flag: 1 => Alle Bits 0

Carry-Flag: Bei Berechnung ist Übertrag entstanden
↳ MSB (Most significant Bit)

Overflow-Flag: Überschreiten des Zahlenbereichs
=> Übertrüge bei Bit 8 & 9 versch. (XOR)

